An `ActorSystem` handles the execution. Akka Streams are, after all, nothing more than actors processing data. In fact, by default, a stream is just one actor. To run the blueprint, you need an `implicit ActorSystem`.

NOTE Don't forget to stop the `ActorSystem` if nothing else needs to run on it. Perhaps you are creating a finite stream in an actor, and the actor should still be running when the stream is finished. This is not the case in the previous example. Nothing else needs to be executed there. If you use the `ScalaTestWithActorTestKit`, it terminates the system for you. If you need to terminate manually in a blueprint that creates a `Future[Done]`, you can use `future.onComplete(_ => system.terminate())`. To run this future, you need an execution context that you can get from the `ActorSystem` with `system.executionContext`.

The materialized values are there so that you can interact with the graph once it has been executed. In this example, the only materialized value is `Future[Done]`. This allows you to wait until the sink completes the future to make the assertion.

Both the source and the flow have materialized values, but they are not used here. There is no interaction with either after the graph is executed. Therefore, they are set to `NotUsed`:

```
val producer: Source[Int, NotUsed] = ...
val processor: Flow[Int, Int, NotUsed] = ...
```

However, in many use cases—especially with infinite streams—you want to interact with a flow or source: for example, to add more values to a stream or cancel it. In either case, if the materialized value is set to something other than `NotUsed`, it can travel through the stream until it reaches the materialization, where you can pull it out of the stream with `.toMat` and assign it to a variable for later use.

The materialization can be on the left or right side of the `.toMat` method. On the left side is `NotUsed`, produced by the previous flow; and on the right side—more precisely, within `.toMat`—is `Future[Done]`, produced by the `Sink.foreach`. The following listing shows the parts of the example: producer, processor, consumer, blueprint, and materialization.

Listing 10.1 Stream components

```
object Main extends App {

  implicit val system = ActorSystem(Behaviors.empty,"runner")              ❶

  var fakeDB: List[Int] = List()
  def storeDB(value: Int) = fakeDB = fakeDB :+ value

  val producer: Source[Int, NotUsed] = Source(List(1, 2, 3))               ❷
  val processor: Flow[Int, Int, NotUsed] = Flow[Int].filter(_ % 2 == 0)    ❸
  val consumer: Sink[Int, Future[Done]] = Sink.foreach(i => storeDB(i))    ❹

  val blueprint: RunnableGraph[scala.concurrent.Future[akka.Done]] =
      producer.via(processor).toMat(consumer)(Keep.right)                  ❺

  val future: Future[Done] = blueprint.run()                               ❻
  future.onComplete( _ => system.terminate)(system.executionContext)       ❼
}
```